

Coding & Solution

Date	3 July 2026
Team ID	xxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Coding & Solution

Repository Details

Attribute	Detail
GitHub URL	https://github.com/saisatyasri/Credit-Card-Approval-Prediction
Primary Language	Python 3.12
Frameworks	Flask 3.1.2 (web), Scikit-learn 1.8.0 + XGBoost 3.2.0 (ML)
Frontend	HTML5, CSS3, JavaScript, Bootstrap 5.3.2
Deployment	Render (free tier) — https://credit-card-approval-prediction-ra22.onrender.com

Setup Instructions

```
# 1. Clone the repository
git clone https://github.com/saisatyasri/Credit-Card-Approval-Prediction.git
cd Credit-Card-Approval-Prediction
```

```
# 2. Install dependencies
pip install -r requirements.txt
```

```
# 3. Download dataset (requires Kaggle API token)
kaggle datasets download -d rikdifos/credit-card-approval-prediction
unzip credit-card-approval-prediction.zip -d data/
```

```
# 4. Train models (generates models/*.pkl)
python train_model.py
```

```
# 5. Run the web application
python app.py
# Visit http://127.0.0.1:5000
```

Key Code Sections

Target Variable Engineering (train_model.py)

BAD_STATUSES = {'1', '2', '3', '4', '5'} # 30+ days past due → Rejected

```
def applicant_label(statuses):
    unique = set(statuses)
    if unique == {'X'}:
        return None # Exclude: no credit history evaluated
    if unique & BAD_STATUSES:
        return 0 # Rejected
    return 1 # Approved

def build_target(credit_df):
    grouped = credit_df.groupby('ID')['STATUS'].apply(list)
    labels = grouped.apply(applicant_label).dropna()
    return labels.astype(int)
```

Feature Engineering (train_model.py)

```
def engineer_features(df):
    df['AGE_YEARS'] = (-df['DAYS_BIRTH'] / 365).round(1)
    employed = df['DAYS_EMPLOYED'].apply(
        lambda x: 0 if x == 365243 else round(-x / 365, 1)
    )
    df['IS_UNEMPLOYED'] = (df['DAYS_EMPLOYED'] == 365243).astype(int)
    df['YEARS_EMPLOYED'] = employed
    df['INCOME_PER_MEMBER'] = (
        df['AMT_INCOME_TOTAL'] / df['CNT_FAM_MEMBERS'].clip(lower=1)
    )
    return df
```

Preprocessing for Inference (app.py)

```
CATEGORICAL_COLS = [
    'NAME_INCOME_TYPE', 'NAME_EDUCATION_TYPE', 'NAME_FAMILY_STATUS',
    'NAME_HOUSING_TYPE', 'OCCUPATION_TYPE'
]
NUMERIC_COLS = [
    'CNT_CHILDREN', 'AMT_INCOME_TOTAL', 'AGE_YEARS', 'YEARS_EMPLOYED',
    'IS_UNEMPLOYED', 'FLAG_MOBIL', 'FLAG_WORK_PHONE', 'FLAG_PHONE',
    'FLAG_EMAIL', 'CNT_FAM_MEMBERS', 'INCOME_PER_MEMBER',
    'CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY'
]
```

```
def preprocess_input(form):
    cat_encoded = [encoder[c].transform([form[c]])[0] for c in CATEGORICAL_COLS]
    num_values = [float(form[c]) for c in NUMERIC_COLS]
```

```
num_scaled = scaler.transform([num_values])[0]
return np.array(cat_encoded + list(num_scaled)).reshape(1, -1)
```

Model Training Summary

Model	Accuracy	ROC-AUC	Class Weight
Logistic Regression	74.3%	0.81	balanced
Decision Tree	76.8%	0.77	balanced
Random Forest	81.91%	0.89	balanced
XGBoost	79.5%	0.87	scale_pos_weight=neg/pos ratio

Best Model: Random Forest — saved as models/model.pkl